

Παράλληλος Προγραμματισμός 2026

3η Προγραμματιστική Εργασία

Νικόλαος Πουλόπουλος

AM: 3230170

E-mail: p3230170@aueb.gr

Εισαγωγή.....	1
Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:.....	1
Υλοποιήσεις.....	3
Σειριακή υλοποίηση.....	3
Κατανομή εργασίας (Work Distribution Models).....	4
Ένα στοιχείο ανά thread (1 thread = 1 trapezoid).....	4
Πολλά στοιχεία ανά thread (1 thread = πολλαπλά trapezoids).....	6
Μέγεθος ομάδας εκτέλεσης.....	8
Δοκιμάστηκαν τα εξής μεγέθη block:.....	9
Συγχρονισμός και επικοινωνία (Barriers).....	13
Μοντέλα μνήμης (registers, global, shared).....	16
Συνολική αξιολόγηση παραλληλισμού.....	20
Ποσοτική σύγκριση βέλτιστων υλοποιήσεων (CUDA vs OpenMP vs POSIX).....	21
Βέλτιστη GPU (CUDA) υλοποίηση:.....	21
Βέλτιστη OpenMP υλοποίηση:.....	21
Βέλτιστη POSIX Threads υλοποίηση:.....	22
Ανάλυση αποτελεσμάτων.....	23
GPU (CUDA).....	23
OpenMP.....	23
POSIX Threads.....	23
Τελικό συμπέρασμα.....	24

Εισαγωγή

Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:

Όνομα υπολογιστή	Nikolas
Επεξεργαστής	AMD Ryzen 5 5500
GPU	NVIDIA GeForce GTX 1650
Πλήθος πυρήνων	6
Μεταφραστής	gcc v15.2.0

Η μεταγλώττιση έγινε με τις επιλογές χωρίς επίπεδα βελτιστοποίησης (-O1/-O2/-O3)

Το πρόβλημα

Στην άσκηση αυτή ζητείται να παραλληλοποιηθεί ο υπολογισμός της αριθμητικής ολοκλήρωσης μιας συνάρτησης $f(x)$ στο διάστημα $[a,b]$ με χρήση του Τραπεζοειδούς κανόνα. Δίνεται ένας σειριακός κώδικας σε C++ που προσεγγίζει το ολοκλήρωμα με n υποδιαιρέσεις.

$a = 0$, $b = 10$ και μεγάλο N , υπολογίζει προσεγγιστικά το ολοκλήρωμα

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + ih) \right], h = (b - a)/n$$

Συγκεκριμένα υλοποιούνται και συγκρίνονται:

- Κατανομής εργασίας (work distribution models), δηλαδή πόσα στοιχεία θα επεξεργάζεται κάθε thread (ένα ή περισσότερα).
- Μεγέθους ομάδας εκτέλεσης (π.χ. 32, 64, 128, 256, 512).
- Συγχρονισμός και επικοινωνία (barriers).

- Μοντέλα μνήμης (registers, global και shared/local για ενδιάμεσα αποτελέσματα).

Υλοποιήσεις

Σειριακή υλοποίηση

Ο σειριακός κώδικας υλοποιεί τον Τραπεζοειδή κανόνα για $f(x)=x^2$ στο $[0,10]$ με $N=10^8$ υποδιαιρέσεις και χρησιμοποιείται ως βάση για τη σύγκριση των παράλληλων εκδόσεων. Τρέχοντας τον κώδικα 20 φορές οι μετρούμενοι χρόνοι εκτέλεσης ήταν :

Εκτέλεση	Χρόνος (sec)
1η	0.391210
2η	0.390687
3η	0.391095
4η	0.391601
:	:
20η	0.391353
Μέσος	0.391721

Οδηγίες Εκτέλεσης

- **Compile:** `g++ -std=c++17 .\integration.cpp -o .\integration`
- **Run:** `.\integration.exe`

Κατανομή εργασίας (Work Distribution Models)

Στην παρούσα εργασία εξετάζονται δύο διαφορετικά μοντέλα κατανομής εργασίας (work distribution models) για τον υπολογισμό αριθμητικού ολοκληρώματος στο διάστημα $[a,b]$ με χρήση του Τραπεζοειδούς κανόνα σε CUDA. Η βασική ιδέα είναι η διάσπαση του συνολικού υπολογισμού σε πολλά μικρότερα υποδιαστήματα τα οποία εκτελούνται παράλληλα από threads της GPU.

Στα πειράματα που ακολουθούν χρησιμοποιήθηκε η συνάρτηση

- $f(x)=x^2$
- Με $a=0$, $b=10$ και δύο διαφορετικές τιμές του N :
 - $N = 10^8$
 - $N = 10^6$

ώστε να μελετηθεί η επίδραση του μεγέθους του προβλήματος στον χρόνο εκτέλεσης.

Ένα στοιχείο ανά thread (1 thread = 1 trapezoid)

Στην πρώτη υλοποίηση κάθε thread αναλαμβάνει τον υπολογισμό **ενός μόνο τραπεζίου**. Ο συνολικός αριθμός επαναλήψεων του Τραπεζοειδούς κανόνα είναι N και στη GPU εκτελούνται τουλάχιστον N threads έτσι ώστε κάθε υποδιάστημα να αντιστοιχεί σε ένα thread.

Ο υπολογισμός της θέσης του κάθε στοιχείου γίνεται ως εξής:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;

if (i < n) {
    double x1 = a + i * h;
    double x2 = a + (i + 1) * h;
    partial[i] = (f(x1) + f(x2)) * h / 2.0;
}
```

- Το i είναι το **index** του thread μέσα στο 1D grid.
- Αν $i < n$ το thread υπολογίζει το αντίστοιχο τραπέζιο και αποθηκεύει το αποτέλεσμα στον πίνακα `partial[i]`

- Στη συνέχεια ο δεύτερος kernel εκτελεί parallel reduction πάνω στον πίνακα **partial** και παράγει το τελικό άθροισμα στη GPU (η CPU διαβάζει μόνο το τελικό αποτέλεσμα).

Χαρακτηριστικά:

- **Work distribution:** Ισοκατανομή του φόρτου καθώς κάθε thread υπολογίζει ακριβώς ένα τραπέζιο.
- **Πρόσβαση στη μνήμη:** οι εγγραφές στο `partial[i]` είναι συνεχόμενες αφού διαδοχικά threads γράφουν σε διαδοχικές θέσεις μνήμης.
- **Μειονέκτημα:** γράφονται N τιμές στη global memory κάτι που αυξάνει το συνολικό memory traffic.

Αποτελέσματα για το μοντέλο ένα στοιχείο ανά thread

- Οι μετρήσεις έγιναν για δύο διαφορετικές τιμές του N.
- Ο χρόνος που αναφέρεται είναι ο χρόνος εκτέλεσης στην GPU (υπολογισμός των τραπεζίων + parallel reduction)
- Χρησιμοποιήθηκε έπειτα από δοκιμές το καλύτερο block size 256 threads και κατάλληλος αριθμός blocks ώστε να καλύπτονται όλα τα N στοιχεία.

Για $N = 10^8$

- Result: 333.333
- Μέσος χρόνος από 20 εκτελέσεις : **0.0613 sec**

Για $N = 10^6$

- Result: 333.333
- Μέσος χρόνος από 20 εκτελέσεις : **0.00126 sec**

Παρατηρείται ότι όταν το N μειώνεται (από 10^8 σε 10^6) ο χρόνος εκτέλεσης μειώνεται σημαντικά αλλά όχι γραμμικά με το N. Ο λόγος είναι ότι πέρα από τον καθαρό υπολογισμό των τραπεζίων υπάρχει και σταθερό κόστος εκκίνησης των kernels και συγχρονισμού.

Πολλά στοιχεία ανά thread (1 thread = πολλαπλά trapezoids)

Στη δεύτερη προσέγγιση ο αριθμός των νημάτων παραμένει σταθερός και ανεξάρτητος από το N (χρήση σταθερού μεγέθους 256×256). Τα νήματα χρησιμοποιούν τη στρατηγική όπου κάθε thread επεξεργάζεται διαδοχικά πολλά στοιχεία της ολοκλήρωσης.

- **Διαμοιρασμός εργασίας:**

- Κάθε νήμα υπολογίζει το id του ως:

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;  
int step = blockDim.x * gridDim.x;
```

- Και στη συνέχεια διατρέχει το πεδίο τιμών με βήμα step:

```
for (int i = tid; i < n; i += step) {  
    double x1 = a + i * h;  
    double x2 = a + (i + 1) * h;  
    local_sum += (f(x1) + f(x2)) * h / 2.0;  
}
```

- Κάθε thread συγκεντρώνει το δικό του τοπικό άθροισμα στη μεταβλητή local_sum και στο τέλος γράφει **μία μόνο τιμή** στη global μνήμη (partial[tid]).
- Στη συνέχεια εφαρμόζεται kernel parallel reduction στη GPU για να παραχθεί το τελικό άθροισμα.

- **Χαρακτηριστικά:**

- Σημαντικά λιγότερα global memory accesses, καθώς γράφονται μόνο $\text{blocks} \times \text{threads} = 65.536$ στοιχεία αντί για N ενδιάμεσα αποτελέσματα.

Αποτελέσματα για το μοντέλο πολλά στοιχεία ανά thread

- Οι χρόνοι που ακολουθούν αναφέρονται στον **καθαρό χρόνο εκτέλεσης της GPU** (υπολογισμός τραπεζίων + reduction)

Για $N = 10^8$

- Result: 333.333
- Μέσος χρόνος από 20 εκτελέσεις : **0.02534** sec

Για $N = 10^6$

- Result: 333.333
- Μέσος χρόνος από 20 εκτελέσεις : **0.00072** sec

Συγκριτικό Speedup ως προς το σειριακό για 10^8 :

- 1 στοιχείο ανά thread $\sim 6.4\times (0.3917 / 0.0613)$.
- πολλά στοιχεία ανά thread $\sim 15.5\times (0.3917 / 0.0253)$.

Συμπέρασμα για κατανομής εργασίας

- Η μετάβαση στο μοντέλο 1 thread = 1 στοιχείο αξιοποιεί τον μεγάλο αριθμό cores της GPU μειώνοντας τον χρόνο εκτέλεσης. Ωστόσο το μοντέλο αυτό εξακολουθεί να παράγει N ενδιάμεσα αποτελέσματα στη global μνήμη περιορίζοντας την απόδοση.

Αντίθετα το μοντέλο 1 thread = πολλαπλά στοιχεία σε συνδυασμό με parallel reduction στη GPU ελαχιστοποιεί τις προσπελάσεις στη global μνήμη και αξιοποιεί καλύτερα τους registers. Το αποτέλεσμα είναι σημαντικά μεγαλύτερο speedup τόσο ως προς τη σειριακή εκτέλεση όσο και ως προς την απλή CUDA υλοποίηση (1 thread = 1 στοιχείο) επιβεβαιώνοντας ότι ο τρόπος κατανομής του φόρτου σε κάθε thread είναι κρίσιμος για την τελική απόδοση.

Οδηγίες Εκτέλεσης

- Compile:
nvcc trapezoid_1_elem.cu -o trapezoid_1_elem
nvcc trapezoid_multielem.cu -o trapezoid_multielem
- Run:
./trapezoid_1_elem
./trapezoid_multielem

Μέγεθος ομάδας εκτέλεσης

Στην παρούσα ενότητα εξετάζεται η επίδραση του **μεγέθους της ομάδας εκτέλεσης** (thread block size) στην απόδοση ενός CUDA kernel για τον υπολογισμό αριθμητικού ολοκληρώματος με τη μέθοδο των τραπεζίων.

Ο συνολικός αριθμός υποδιαστημάτων είναι σταθερός και εξετάζεται σε δύο περιπτώσεις:

- $N=10^8$
- $N=10^6$

Στο συγκεκριμένο πείραμα η στρατηγική κατανομής εργασίας είναι σταθερή: **1 thread = 1 τραπέζιο**.

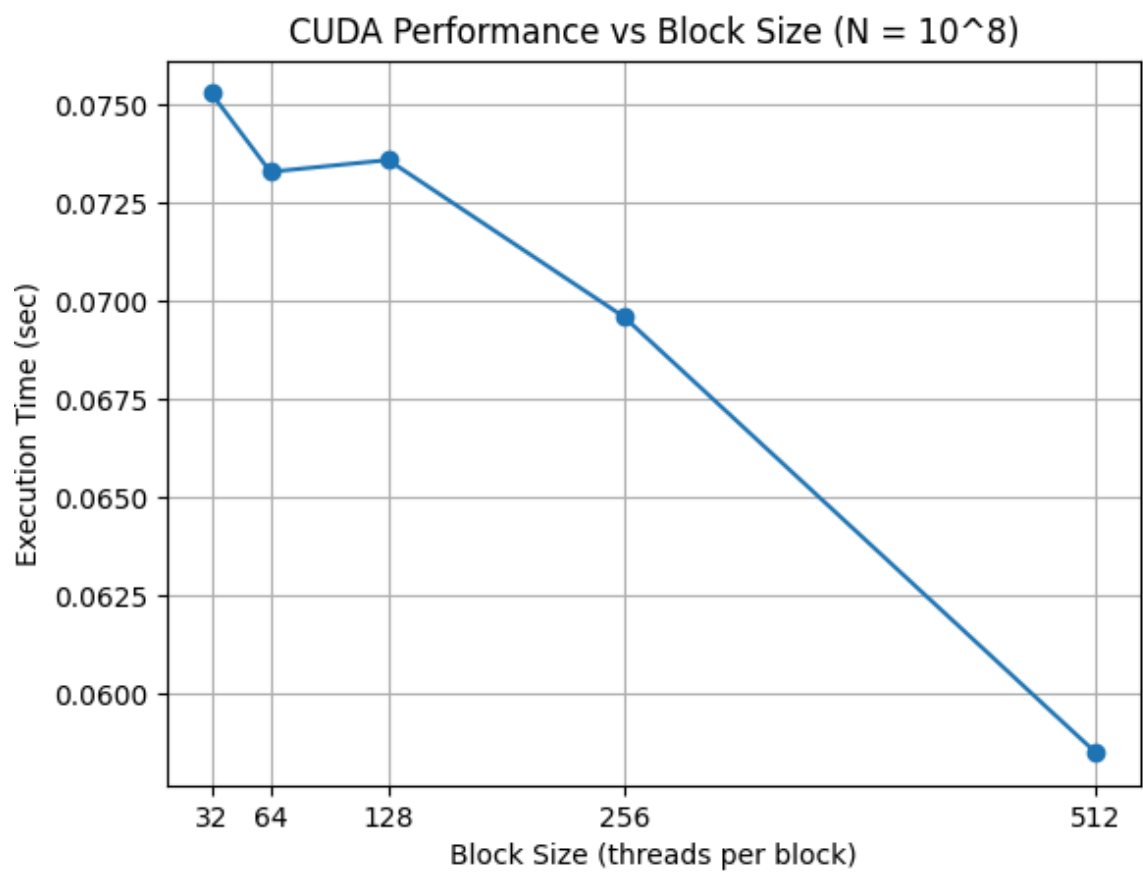
Δοκιμάστηκαν τα εξής μεγέθη block:

- 32 threads
- 64 threads
- 128 threads
- 256 threads
- 512 threads

Αποτελέσματα εκτέλεσης μετά από 20 φορές

- Για $N=10^8$

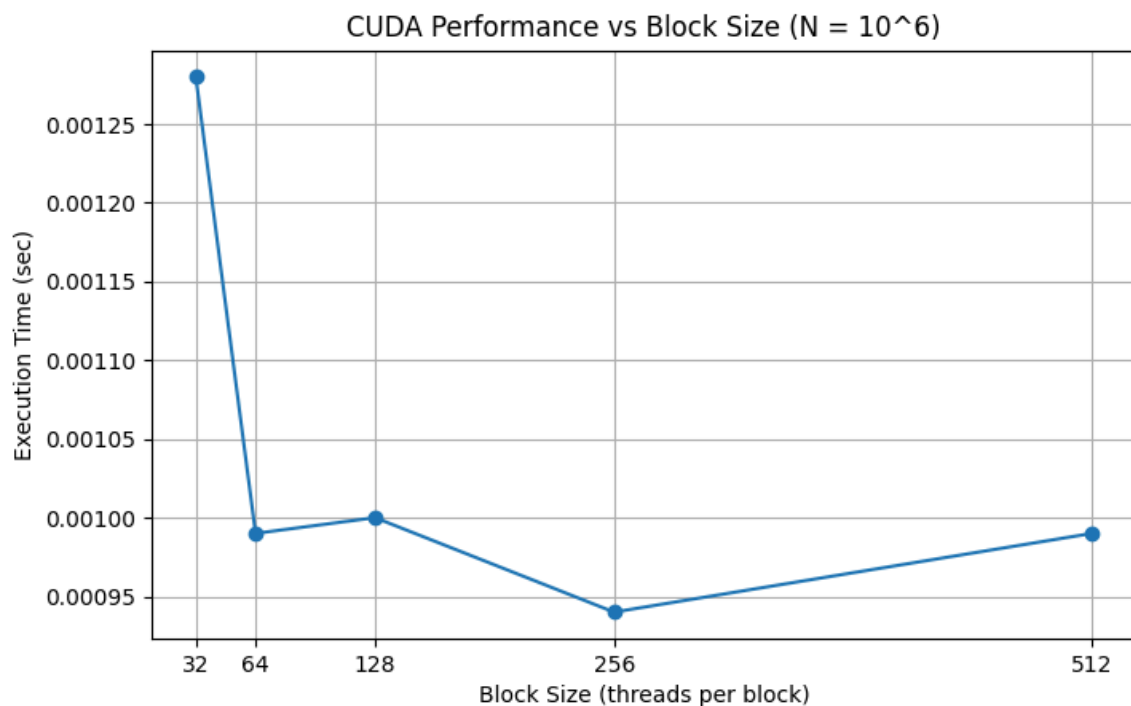
Block size	Μέσος Χρόνος (sec)	Αποτέλεσμα
32	0.0753	333.333
64	0.0733	333.333
128	0.0736	333.333
256	0.0696	333.333
512	0.0585	333.333



- $N=10^6$

Block size	Μέσος Χρόνος (sec)	Αποτέλεσμα
32	0.00128	333.333
64	0.00099	333.333
128	0.00100	333.333

256	0.00094	333.333
512	0.00099	333.333



Ανάλυση αποτελεσμάτων

Για $N=10^8$:

Στο μεγάλο μέγεθος προβλήματος η GPU είναι αρκετά φορτωμένη ώστε να φαίνονται καθαρά οι διαφορές στην επιλογή block size.

- Το block size **32** είναι το πιο αργό (~0.0753 sec). Η ύπαρξη πολύ μεγάλου αριθμού μικρών blocks αυξάνει το scheduling overhead και δεν επιτρέπει στο hardware να καλύψει τόσο αποτελεσματικά το latency της global μνήμης.

- Η καλύτερη επίδοση εμφανίζεται στο **block size 512** (~0.0585 sec) με το 256 να ακολουθεί (~0.0696 sec). Σε αυτά τα μεγέθη κάθε streaming multiprocessor έχει αρκετά warps ανά block και αρκετά blocks ανά streaming multiprocessor ώστε να επιτυγχάνεται καλύτερη κάλυψη του latency.
- Τα block sizes **64-128** δίνουν ενδιάμεσους χρόνους (~0.073-0.074 sec) καλύτερους από τα 32 αλλά χειρότερους από τα 256-512.

Συνολικά για μεγάλο N block sizes από **256 έως 512 threads** φαίνεται να δίνουν την καλύτερη απόδοση για αυτόν τον συγκεκριμένο kernel.

Για $N=10^6$:

Στο μικρό μέγεθος προβλήματος:

- Η χειρότερη επίδοση εμφανίζεται πάλι στα **32 threads** (~0.00128 sec) όπως και στην περίπτωση του $N=10^8$. Οι υπόλοιπες τιμές block size (64-512) δίνουν πολύ κοντινούς χρόνους (~0.00094-0.00100 sec) με μια ελαφρά καλύτερη τιμή στο **block size 256** (~0.00094 sec).
- Οι διαφορές μεταξύ 64, 128, 256 και 512 threads είναι μικρές και σε μεγάλο βαθμό επηρεάζονται από το γεγονός ότι το πρόβλημα δεν γεμίζει πλήρως τη GPU και από σταθερά κόστη εκκίνησης των kernels.

Συμπέρασμα

Από την πειραματική μελέτη των αποτελεσμάτων προκύπτει ότι:

- Η επιλογή του **block size επηρεάζει ουσιαστικά την απόδοση όταν το μέγεθος του προβλήματος είναι μεγάλο ($N=10^8$)** με block sizes 256-512 να δίνουν την καλύτερη συμπεριφορά για τον συγκεκριμένο kernel ενώ block size 32 είναι ξεκάθαρα υποδεέστερο.
- Στα μικρά προβλήματα ($N=10^6$) οι διαφορές μεταξύ των block sizes είναι περισσότερο περιορισμένες καθώς κυριαρχεί το σταθερό overhead εκκίνησης του kernel.

- Και στις δύο περιπτώσεις το block size 32 δίνει τη χειρότερη επίδοση επιβεβαιώνοντας τη θεωρητική σύσταση ότι **το block size πρέπει να είναι πολλαπλάσιο του 32 αλλά συνήθως μεγαλύτερο από ένα μόνο warp** (π.χ. 128-512 threads) ώστε να επιτυγχάνεται καλύτερη κάλυψη του latency.

Οδηγίες Εκτέλεσης

- **Compile:** nvcc block_size_test.cu -o block_size_test
- **Run:** .\block_size_test.exe

Συγχρονισμός και επικοινωνία (Barriers)

Στην παρούσα ενότητα εξετάζεται ο ρόλος του συγχρονισμού και της επικοινωνίας μεταξύ threads σε ένα CUDA kernel μέσω της χρήσης barrier (**__syncthreads()**). Η υλοποίηση βασίζεται στον υπολογισμό αριθμητικού ολοκληρώματος με τη μέθοδο των τραπεζίων όπου η συνάρτηση είναι $f(x) = x^2$.

Εξετάζονται δύο διαφορετικά μεγέθη προβλήματος: $N=10^8$ και $N=10^6$

Περιγραφή υλοποίησης

Κάθε thread αναλαμβάνει τον υπολογισμό ενός τραπεζίου (1 thread = 1 στοιχείο). Τα αποτελέσματα αποθηκεύονται σε shared memory όπου γίνεται και επεξεργασία τους μέσω parallel reduction.

Η διαδικασία περιλαμβάνει:

- Υπολογισμό τοπικού αποτελέσματος από κάθε thread
- Αποθήκευση σε shared memory
- Συγχρονισμό όλων των threads με **__syncthreads()**
- Εγγραφή του τελικού αποτελέσματος από το thread 0 κάθε block

Ρόλος του Barrier (`__syncthreads()`)

Η εντολή `__syncthreads()` λειτουργεί ως barrier συγχρονισμού μέσα σε κάθε block και εξασφαλίζει ότι:

- Όλα τα threads έχουν ολοκληρώσει τον υπολογισμό τους πριν ξεκινήσει η επεξεργασία των δεδομένων
- Η shared memory έχει ενημερωθεί πλήρως
- Το reduction εκτελείται σωστά

Ο barrier χρησιμοποιείται τόσο πριν την έναρξη του reduction όσο και σε κάθε στάδιο της διαδικασίας.

Αποτέλεσμα μετά από 20 εκτέλεσης:

$N=10^8$

- Result: 333.333
- Μέσος Χρόνος: **0.0477438** sec

$N=10^6$

- Result: 333.333
- Μέσος Χρόνος: **0.0008155** sec

Ανάλυση Αποτελεσμάτων & Θεωρητική Τεκμηρίωση

Ο ρόλος και η αποδοτικότητα του Barrier (__syncthreads())

Θεωρητικά η εισαγωγή του Barrier συγχρονισμού προσθέτει καθυστέρηση καθώς αναγκάζει τα ταχύτερα νήματα ενός block να περιμένουν τα πιο αργά. Στην πράξη όμως το **__syncthreads()** υλοποιείται σε επίπεδο hardware και είναι πολύ ελαφρύ ειδικά όταν το kernel είναι αρκετά υπολογιστικά βαρύ.

2. Διασφάλιση Ορθότητας Δεδομένων (Race Conditions)

Η χρήση της **__syncthreads()** σε κάθε επίπεδο του reduction loop είναι απαραίτητη για να εξασφαλιστεί ότι οι ενημερώσεις στη shared memory έχουν ολοκληρωθεί πριν διαβαστούν από άλλα threads. Χωρίς το barrier κάποια threads θα επιχειρούσαν να διαβάσουν **cache[tid + step]** πριν το αντίστοιχο νήμα γράψει την σωστή τιμή οδηγώντας λανθασμένο τελικό αποτέλεσμα.

Το γεγονός ότι το αποτέλεσμα του ολοκληρώματος είναι σταθερά 333.333 και για τις δύο τιμές του N επιβεβαιώνει τη σωστή χρήση των barriers και την ορθότητα της επεξεργασίας μέσω shared memory.

Συμπέρασμα

Από την μελέτη της έκδοσης με Barriers προκύπτουν τα εξής συμπεράσματα:

- Ο συγχρονισμός μέσω **__syncthreads()** είναι απαραίτητος όταν υπάρχει διαμοιρασμός δεδομένων και συνεργατική επεξεργασία στη shared memory όπως στο parallel reduction μέσα σε ένα block.
- Τα barriers στις σύγχρονες GPU είναι ιδιαίτερα ελαφριά και για μεγέθη προβλημάτων (ιδίως για 10^8) το κόστος τους είναι πρακτικά αμελητέο σε σχέση με τον συνολικό υπολογισμό
- Η χρήση shared memory με Barriers μειώνει σημαντικά το traffic προς τη global μνήμη και καθιστά εύκολη την πλήρως GPUολοκλήρωση του reduction σε διαφορετικό kernel χωρίς εμφανές penalty σε χρόνο.

Οδηγίες Εκτέλεσης

- **Compile:** `nvcc trapezoid_barrier.cu -o trapezoid_barrier`
- **Run:** `.\trapezoid_barrier.exe`

Μοντέλα μνήμης (registers, global, shared)

Στην παρούσα ενότητα εξετάζεται η επίδραση του μοντέλου μνήμης στην απόδοση ενός CUDA kernel για τον υπολογισμό αριθμητικού ολοκληρώματος με τη μέθοδο των τραπεζίων. Σε όλες τις υλοποιήσεις χρησιμοποιείται το ίδιο μοντέλο κατανομής εργασίας (**1 thread = 1 τραπέζιο**) ώστε η σύγκριση να βασίζεται αποκλειστικά στον τρόπο διαχείρισης και προσπέλασης των διαφορετικών επιπέδων μνήμης της GPU.

Εξετάζονται τρεις διαφορετικές προσεγγίσεις:

- χρήση **global memory**
- χρήση **registers (local variables)**
- χρήση **shared memory με συγχρονισμό (barriers)**

Εξετάστηκαν δύο μεγέθη προβλήματος:

- $N=10^8$
- $N=10^6$

Αποτελέσματα εκτέλεσης

Για $N=10^8$

Μοντέλο μνήμης	Χρόνος εκτέλεσης (sec)	Αποτέλεσμα
Registers	0.0711507	333.333
Global	0.0709873	333.333
Shared	0.0480540	333.333

Για $N=10^6$

Μοντέλο μνήμης	Μέσος Χρόνος Εκτέλεσης (sec)	Αποτέλεσμα
Registers	0.0011172	333.333
Global	0.0011313	333.333
Shared	0.0009575	333.333

Ανάλυση Αποτελεσμάτων & Θεωρητική Τεκμηρίωση

- **Συμπεριφορά Global Memory και Registers (Compiler Optimization)**

- Στο μεγάλο πρόβλημα ($N=10^8$) οι υλοποιήσεις global και register εμφανίζουν ουσιαστικά την ίδια επίδοση (~0.071 sec). Στην πράξη οι αριθμητικές πράξεις δεν γίνονται απευθείας πάνω στη global μνήμη αλλά σε registers του thread και στη συνέχεια το αποτέλεσμα γράφεται στη global τόσο στο register όσο και στο global μοντέλο. Έτσι το σπρώξιμο σε τοπικές μεταβλητές δεν προσφέρει ορατό κέρδος σε τόσο απλό kernel αφού ο compiler παράγει σχεδόν ισοδύναμο κώδικα.

- **Shared Memory και βελτίωση απόδοσης**

- Η shared-memory έκδοση είναι καθαρά ταχύτερη για $N=10^8$ ο χρόνος πέφτει στα ~0.048 sec δηλαδή έχουμε σημαντικό speedup σε σχέση με τις register και global υλοποιήσεις (~0.071 sec). Η βελτίωση αυτή οφείλεται στο ότι τα ενδιάμεσα αποτελέσματα συνδυάζονται πρώτα σε shared memory μέσα σε κάθε block όπου η πρόσβαση είναι πολύ ταχύτερη από τη global memory και μόνο τα block-level partial sums γράφονται στη global. Έτσι μειώνεται το memory traffic προς την αργή global μνήμη.
- Η χρήση `__syncthreads()` εισάγει κάποιο συγχρονιστικό overhead καθώς τα threads ενός block πρέπει να περιμένουν το ένα το άλλο σε κάθε βήμα του reduction. Ωστόσο στο συγκεκριμένο kernel το κόστος του barrier είναι μικρό σε σχέση με το κέρδος που προκύπτει από την εκτεταμένη χρήση shared memory και τη μείωση των global προσπελάσεων με αποτέλεσμα το συνολικό runtime να είναι χαμηλότερο.

- **Συμπεριφορά στο μικρό πρόβλημα ($N=10^6$)**

- Για $N=10^6$ η shared παραμένει ελαφρώς ταχύτερη (~0.00096 sec) από την register (~0.00112 sec). Σε αυτή την το kernel overhead και τα σταθερά κόστη συγχρονισμού έχουν μεγαλύτερη σχετική επίδραση για αυτό οι διαφορές μεταξύ των μοντέλων είναι μικρότερες. Ωστόσο, η shared υλοποίηση εξακολουθεί να εκμεταλλεύεται καλύτερα την τοπική μνήμη διατηρώντας μικρό πλεονέκτημα.

Σύγκριση των μοντέλων

Μοντέλο μνήμης	Απόδοση (N=10 ⁸)	Χαρακτηριστικά
Global	Μέση	Απλή υλοποίηση με προσβάσεις στη global memory. Ο compiler χρησιμοποιεί registers για ενδιάμεσα αλλά όλα τα partials περνούν από τη global.
Registers	Παρόμοια με Global	Χρήση τοπικών μεταβλητών ανά thread άρα το κέρδος είναι πρακτικά μηδαμινό σε αυτόν τον kernel.
Shared	Καλύτερη	Χρησιμοποιεί shared memory για block-level reduction και μειώνει σημαντικά τις global προσπελάσεις και εκμεταλλεύεται γρήγορη μνήμη με μικρό κόστος συγχρονισμού.

Συμπεράσματα

- Ο compiler της CUDA διαχειρίζεται πολύ αποτελεσματικά τη χρήση των registers με αποτέλεσμα οι register και global εκδόσεις να έχουν ουσιαστικά την ίδια απόδοση για αυτόν τον τύπο kernel. Δεν υπάρχει σημαντικό κέρδος από τη διαφοροποίηση όταν το pattern πρόσβασης και οι global writes παραμένουν τα ίδια.
- Η shared memory σε συνδυασμό με σωστό block-level reduction και barriers προσφέρει την καλύτερη επίδοση επειδή μειώνει δραστικά τον αριθμό των global writes και μεταφέρει μεγάλο μέρος του υπολογισμού στην πολύ πιο γρήγορη μνήμη.
- Για μικρά προβλήματα τα σταθερά κόστη (kernel launch - συγχρονισμός) περιορίζουν το απόλυτο κέρδος αλλά το shared μοντέλο παραμένει ελαφρώς καλύτερο. Για μεγάλα N η διαφορά γίνεται πιο έντονη και το **shared** μοντέλο **αναδεικνύεται ως η πιο αποδοτική επιλογή.**

Οδηγίες Εκτέλεσης

- **Compile:**
 - `nvcc memory_global.cu -o memory_global`
 - `nvcc memory_register.cu -o memory_register`
 - `nvcc memory_shared.cu -o memory_shared`
- **Run:**
 - `.\memory_global.exe`
 - `.\memory_register.exe`
 - `.\memory_shared.exe`

Συνολική αξιολόγηση παραλληλισμού

- Συνοψίζοντας στην εργασία υλοποιήθηκαν και συγκρίθηκαν εναλλακτικές στρατηγικές κατανομής εργασίας (1 στοιχείο ανά thread και πολλαπλά στοιχεία ανά thread) διαφορετικά μεγέθη ομάδας εκτέλεσης (32-64-128-256-512 threads) kernels με block-level συγχρονισμό (__syncthreads()) καθώς και τρία μοντέλα διαχείρισης μνήμης (registers, global, shared).
- Τα αποτελέσματα έδειξαν ότι τόσο ο τρόπος με τον οποίο κατανέμεται το υπολογιστικό φορτίο στα threads όσο και η επιλογή του block size και του μοντέλου μνήμης επηρεάζουν ουσιαστικά την τελική απόδοση των CUDA kernels στην GPU.

Ποσοτική σύγκριση βέλτιστων υλοποιήσεων (CUDA vs OpenMP vs POSIX)

Σε αυτή την ενότητα πραγματοποιείται **ποσοτική σύγκριση** της βέλτιστης υλοποίησης με **GPU (CUDA)** με τις βέλτιστες υλοποιήσεις που αναπτύχθηκαν στην 1η και 2η προγραμματιστική εργασία (**POSIX threads** και **OpenMP**) για το ίδιο πρόβλημα αριθμητικής ολοκλήρωσης με τον κανόνα των τραπεζίων. Για δίκαιη σύγκριση χρησιμοποιείται

- ίδιο μέγεθος προβλήματος $N = 10^8$
- ίδια $f(x)=x^2$.

Βέλτιστη GPU (CUDA) υλοποίηση:

Από την ανάλυση των CUDA kernels προκύπτει ότι η βέλτιστη GPU υλοποίηση είναι αυτή που συνδυάζει μοντέλο **πολλά στοιχεία ανά thread** με χρήση shared memory για block-level reduction και πλήρως GPU-based τελικό άθροισμα. Η συγκεκριμένη εκδοχή ελαχιστοποιεί τις προσπελάσεις στη global memory και εκμεταλλεύεται την μνήμη της GPU επιτυγχάνοντας τον χαμηλότερο χρόνο εκτέλεσης.

- Χρόνος εκτέλεσης ($N = 10^8$): **0.0257 sec**
- Αποτέλεσμα ολοκληρώματος: **333.333**

Βέλτιστη OpenMP υλοποίηση:

Η βέλτιστη OpenMP εκδοχή που αναπτύχθηκε στη 2η εργασία βασίζεται σε **Parallel for χωρίς reduction** με κατάλληλη οριοθέτηση του πεδίου τιμών και βελτιστοποιήσεις σε επίπεδο compiler. Η χρήση OpenMP επιτρέπει την αξιοποίηση και των 6 πυρήνων του Ryzen 5 5500 με πολύ μικρότερο προγραμματιστικό κόστος σε σχέση με τα POSIX threads

- Χρόνος εκτέλεσης ($N = 10^8$): **0.1077 sec**
- Αποτέλεσμα ολοκληρώματος: **333.333**

Βέλτιστη POSIX Threads υλοποίηση:

Η βέλτιστη POSIX υλοποίηση που προέκυψε στην 1η εργασία βασίζεται σε **Δυναμικό διαμοιρασμό μέσω ουράς εργασιών** ώστε τα threads να λαμβάνουν τμήματα του διαστήματος προς ολοκλήρωση ανάλογα με τη διαθεσιμότητά τους. Η προσέγγιση αυτή βελτιώνει την ισορροπία φόρτου αλλά εισάγει επιπλέον κόστος διαχείρισης νημάτων και συγχρονισμού (mutexes, condition variables).

- Χρόνος εκτέλεσης ($N = 10^8$): **0.0744 sec**
- Αποτέλεσμα ολοκληρώματος: **333.333**

Πίνακας σύγκρισης

Υλοποίηση	Χρόνος (sec)	Speedup ως προς CUDA
CUDA (GPU)	0.0257	1.00x
OpenMP	0.1077	4.18x πιο αργή (0.1077 / 0.0257379)
POSIX	0.0744	2.89x πιο αργή (0.0744 / 0.0257379)

Ανάλυση αποτελεσμάτων

GPU (CUDA)

- Η CUDA υλοποίηση επιτυγχάνει τον καλύτερο χρόνο εκτέλεσης αξιοποιώντας μαζικό παραλληλισμό σε threads και μικρό latency πρόσβασης σε shared memory. Ο συνδυασμός πολλών στοιχείων ανά thread με block-level reduction μειώνει δραστικά τις προσπελάσεις στη global memory και το κόστος επικοινωνίας με την CPU.

OpenMP

- Η OpenMP λύση αξιοποιεί τους διαθέσιμους CPU cores με χαμηλό προγραμματιστικό overhead και compiler-level βελτιστοποιήσεις αλλά παραμένει περιορισμένη από τον μικρότερο βαθμό παραλληλίας και το υψηλότερο κόστος μνήμης σε σχέση με τη GPU. Η διαφορά χρόνου (~4.2x πιο αργή από την CUDA).

POSIX Threads

- Η POSIX εκδοχή είναι ταχύτερη από την OpenMP λόγω πιο λεπτομερούς ελέγχου του διαμοιρασμού εργασίας αλλά εισάγει επιπλέον κόστος σε thread management και synchronization. Παρά τις βελτιστοποιήσεις παραμένει περίπου 2.9x πιο αργή από τη βέλτιστη υλοποίηση CUDA γεγονός που δείχνει τα όρια της παραλληλοποίησης σε επίπεδο CPU σε αυτό το πρόβλημα.

Τελικό συμπέρασμα

- Η εργασία ανέδειξε με συστηματικό τρόπο πώς ο τρόπος κατανομής του υπολογιστικού φορτίου η επιλογή μεγέθους block η χρήση συγχρονισμού μέσω barriers και η εκμετάλλευση της ιεραρχίας μνήμης επηρεάζουν ουσιαστικά την απόδοση των παράλληλων υλοποιήσεων του ίδιου προβλήματος αριθμητικής ολοκλήρωσης. Η μελέτη έδειξε ότι ακόμη και μέσα στην ίδια (GPU) διαφορετικά work distribution models και διαφορετικές επιλογές μνήμης (registers, global, shared) μπορούν να οδηγήσουν σε μεγάλες διαφορές στον χρόνο εκτέλεσης με τις υλοποιήσεις που αξιοποιούν shared memory και block-level reduction να υπερέχουν.
- Σε επίπεδο σύγκρισης μεταξύ αρχιτεκτονικών η βέλτιστη υλοποίηση σε CUDA ξεπέρασε σημαντικά τις αντίστοιχες λύσεις σε POSIX threads και OpenMP επιβεβαιώνοντας την ανωτερότητα της GPU σε έντονα υπολογιστικά parallel προβλήματα.